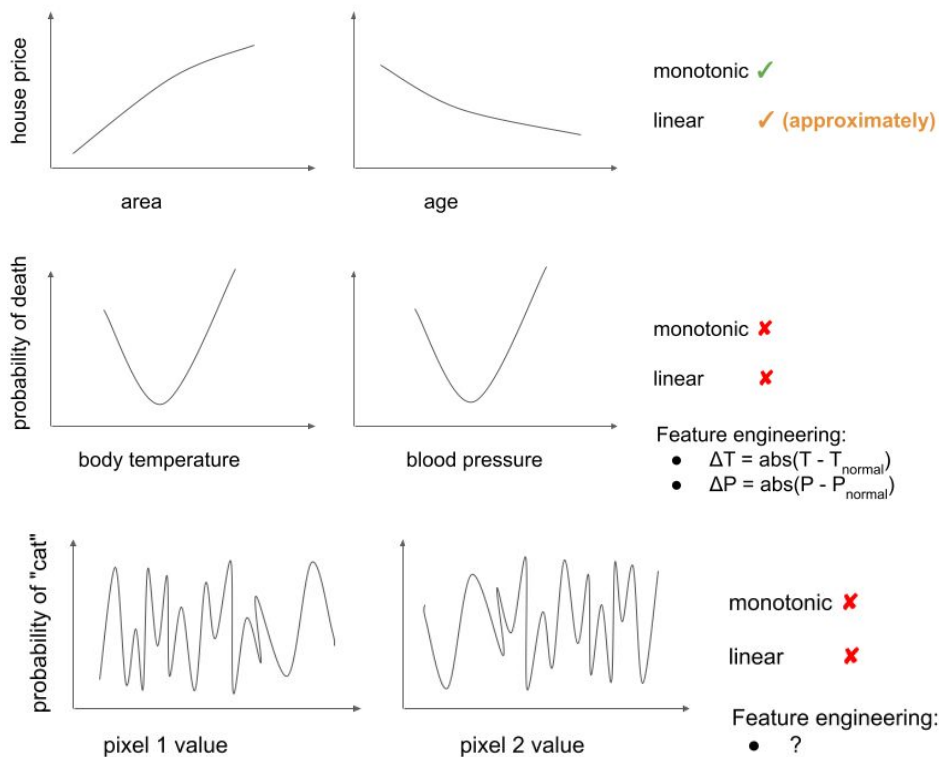


# Generalized Linear Regression

Hrachya Asatryan

# Linearity

## Linear Models May Go Wrong



# Simple Regression Model

## The simplest linear model for regression is:

A linear combination of the input variables  $\mathbf{x} = (x_1, \dots, x_D)^T$ :

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + \dots + w_Dx_D$$

The key property of this model is that **it is a linear function of the parameters  $w_0, \dots, w_D$**

As  $y(\mathbf{x}, \mathbf{w})$  is a linear combination of input variables  $x$ , this largely limits the model, thus we can introduce a basis function from  $\mathbf{x}$ ,  $\phi_j(\mathbf{x})$ , extending the class of models by considering linear combinations of fixed nonlinear functions of the input variables. Thus  $y(\mathbf{x}, \mathbf{w})$  can be given as:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

where  $\mathbf{w} = (w_0, \dots, w_{M-1})^T$  and  $\boldsymbol{\phi} = (\phi_0, \dots, \phi_{M-1})^T$ , here we introduce a  $\phi_0(\mathbf{x}) = 1$  dummy function to incorporate for the  $w_0$  bias parameter.

# Basis function

## What can be our basis function?

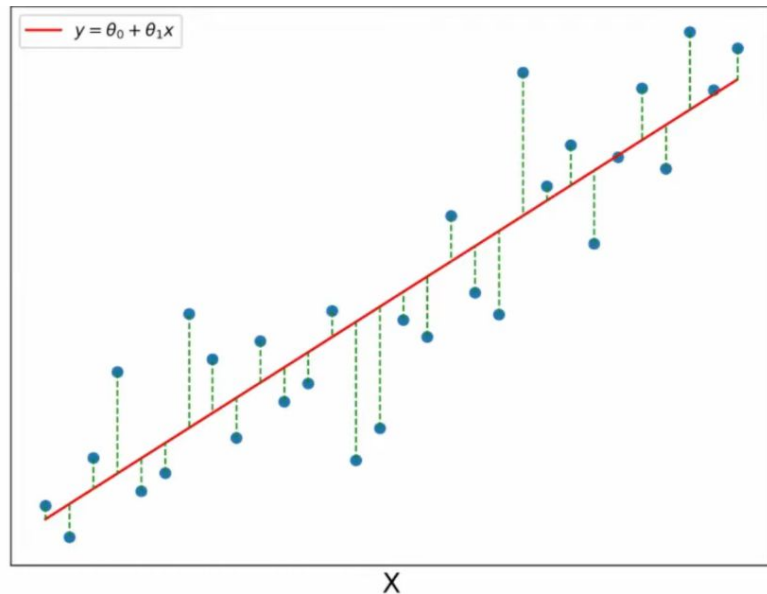
- Polynomial function, such as  $\varphi_j(x) = x^j$
- Gaussian basis functions

$$\phi_j(x) = \exp \left\{ -\frac{(x - \mu_j)^2}{2s^2} \right\}$$

- Sigmoidal basis function

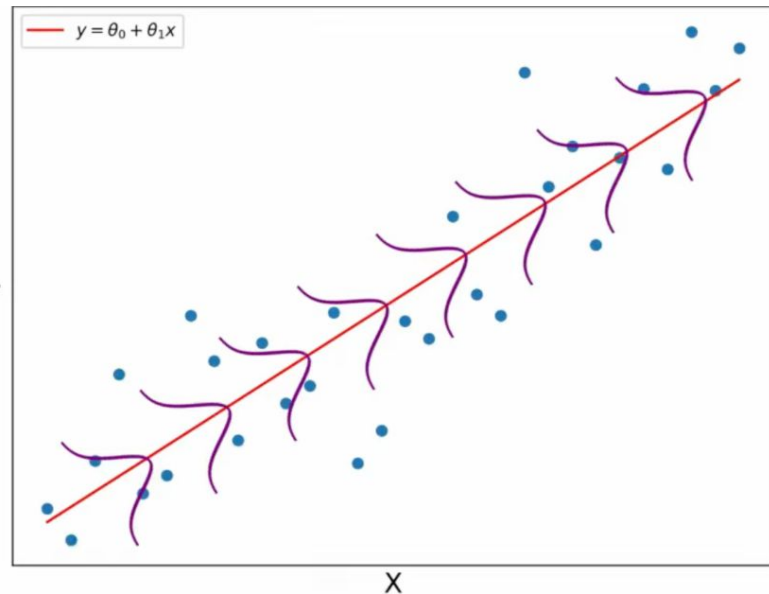
$$\phi_j(x) = \sigma \left( \frac{x - \mu_j}{s} \right)$$

# Fitting the model



Least Squares

same?  
↔



Maximum likelihood

# Maximum likelihood

## Maximum likelihood:

Let's assume that each point is drawn from a Gaussian distribution, with the mean being the regression line itself. This can be illustrated mathematically as:

$$t = y(\mathbf{x}, \mathbf{w}) + \epsilon$$

Where  $\epsilon$  is a zero mean Gaussian random variable with precision (inverse variance)  $\beta$ . Thus we can write:

$$p(t|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \beta^{-1})$$

In case of a squared loss function, the optimal prediction for a new value of  $x$  will be given by the expected value of the target variable. In case of the Gaussian distribution written above, we have:

$$\mathbb{E}[t|\mathbf{x}] = \int t p(t|\mathbf{x}) dt = y(\mathbf{x}, \mathbf{w})$$

Note that we solve this case assuming that the conditional distribution of  $t$  given  $x$  is unimodal.

# Maximum likelihood

## Maximum likelihood:

We group the target variables  $\{t_n\}$  into a column vector that we denote by  $\mathbf{t}$ . Making the assumption that these data points are drawn independently from the distribution, we obtain the following expression for the likelihood function, which is a function of the adjustable parameters  $\mathbf{w}$  and  $\beta$ , in the form:

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \beta^{-1})$$

As we are not going to model the distribution of the input variables, we'll drop the  $\mathbf{x}$ . Taking the logarithm of the both sides, we get:

$$\ln p(\mathbf{t}|\mathbf{w}, \beta) = \sum_{n=1}^N \ln \mathcal{N}(t_n | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \beta^{-1}) = \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \beta E_D(\mathbf{w})$$

With the sum-of-squares error function is defined as:  $E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)\}^2$ .

# Maximum likelihood

## Maximum likelihood:

Now we can use maximum likelihood to determine  $\mathbf{w}$  and  $\beta$ . First let's maximize with the respect to  $\mathbf{w}$ . Maximization of the likelihood function for a linear model is equivalent to minimizing the sum-of-squares error function. The gradient of the log likelihood function, takes the form of:

$$\nabla \ln p(\mathbf{t}|\mathbf{w}, \beta) = \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\} \phi(\mathbf{x}_n)^T$$

Setting the gradient to zero we get:

$$0 = \sum_{n=1}^N t_n \phi(\mathbf{x}_n)^T - \mathbf{w}^T \left( \sum_{n=1}^N \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T \right).$$

Solving this for  $\mathbf{w}$  we obtain:

$$\mathbf{w}_{\text{ML}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

Which is called the **normal equation for linear regression**.



# The design matrix

## What is $\Phi$ ?

Here  $\Phi$  is an  $N \times M$  matrix, called the design matrix, whose elements are given by  $\Phi_{nj} = \phi_j(\mathbf{x}_n)$ , so that:

$$\Phi = \begin{pmatrix} \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \cdots & \phi_{M-1}(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \phi_1(\mathbf{x}_2) & \cdots & \phi_{M-1}(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_N) & \phi_1(\mathbf{x}_N) & \cdots & \phi_{M-1}(\mathbf{x}_N) \end{pmatrix}$$

We can introduce the Moore-Penrose pseudo-inverse of the matrix  $\Phi$  as a generalization of the notion of matrix inverse to non square matrices.

$$\Phi^\dagger \equiv (\Phi^T \Phi)^{-1} \Phi^T$$

# Role of the bias term

## Insight into the role of the bias:

If we make the bias parameter( $w_0$ ) explicit, then the error function becomes:

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left\{ t_n - w_0 - \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}_n) \right\}^2$$

Setting the derivative with respect to  $w_0$  equal to zero, and solving for  $w_0$ , we obtain:

$$\bar{t} = \frac{1}{N} \sum_{n=1}^N t_n, \quad \overline{\phi_j} = \frac{1}{N} \sum_{n=1}^N \phi_j(\mathbf{x}_n) \quad w_0 = \bar{t} - \sum_{j=1}^{M-1} w_j \overline{\phi_j}$$

Thus the bias  $w_0$  compensates for the difference between the averages (over the training set) of the target values and the weighted sum of the averages of the basis function values.

# Precision

## Precision:

We can also maximize the log likelihood function with respect to the noise precision parameter  $\beta$ , giving:

$$\frac{1}{\beta_{\text{ML}}} = \frac{1}{N} \sum_{n=1}^N \{t_n - \mathbf{w}_{\text{ML}}^T \phi(\mathbf{x}_n)\}^2$$

and so we see that the inverse of the noise precision is given by the residual variance of the target values around the regression function.

# Problem Formulation

## What do we have?

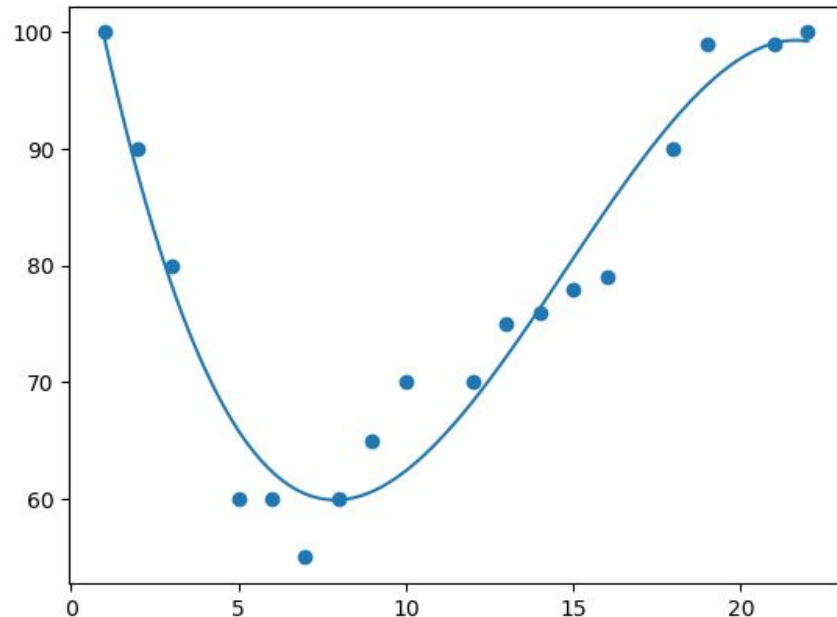
A training data set comprising of  $\mathbf{N}$  observations  $\{x_n\}$ , where  $n = 1, \dots, N$ , together with corresponding target values  $\{t_n\}$ .

## What is our goal?

To predict the value of  $\mathbf{t}$  for a new value of  $\mathbf{x}$ .

## How do we do that?

We assume that  $\mathbf{t}$  can be modeled as a nonlinear but smooth function of  $\mathbf{x}$  — specifically, a polynomial of fixed degree. This way we can capture curved relationships in the data!



# Polynomial Regression Model

## How to move beyond linearity:

- Linear regression assumes a **linear connection** between  $\mathbf{x}$  and  $\mathbf{t}$ .
- In the real world it is extremely unlikely that the true function  $f(\mathbf{x})$  is actually linear in  $\mathbf{x}$ .
- The main idea of **polynomial regression** to add new variables based on the original input  $\mathbf{x}$  ( $\mathbf{x}^2$ ,  $\mathbf{x}^3$ , etc) and apply a linear model to these new features.
- This allows modeling more complex (nonlinear) relationships while still using simple linear methods.
- Let's denote the  $m$ -th ( $m=1,\dots,M$ ) transformation of  $X$  as  $h_m(X)$  and introduce a weight variable  $\beta_m$  thus we will have:

$$f(X) = \sum_{m=1}^M \beta_m h_m(X)$$

$f(X)$  is a linear basis expansion in basis of  $X$ . Once the basis functions  $h_m(X)$  have been determined, the models are linear in these new variables, and the fitting proceeds as before!

# Polynomial Regression Model

## Design Matrix

- If we choose our basis functions as polynomials, then our design matrix  $\Phi$  becomes the Vandermonde matrix:

$$V = V(x_0, x_1, \dots, x_m) = \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^n \end{pmatrix}$$

## Interpolation Theorem:

If we have  $N$  distinct points  $\{x_i\}$ , there exists a polynomial of degree at most  $N - 1$  such that:

$$\hat{y}(x_i) = y_i \quad \forall i$$

# Polynomial Regression Model

## Ill-Conditioning

In polynomial regression:

$$\Phi = \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^n \end{pmatrix}$$

Columns are highly correlated: if  $x \in [0,1]$

$$x^5 \approx x^4 \approx x^3$$

Columns become nearly linearly dependent  $\Rightarrow \Phi^T \Phi$  is nearly singular.

As a consequence, even if invertible,  $(\Phi^T \Phi)^{-1}$  may be numerically unstable.

# Polynomial Regression Model

## Condition Number

Let  $A$  be a matrix. The condition number is defined as:

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

where:

- $\sigma_{\max}$  = largest singular\* value
- $\sigma_{\min}$  = smallest singular\* value

## Interpretation

- $\kappa(A) \approx 1 \rightarrow$  well-conditioned
- Large  $\kappa(A) \rightarrow$  ill-conditioned
- If  $\sigma_{\min} \rightarrow 0$ , then  $\kappa(A) \rightarrow \infty$

\*Singular values of a matrix are the non-negative square roots of the eigenvalues of  $A^T A$  (or  $AA^T$ ).



# Polynomial Regression Model

## Condition Number

### Why It Matters

Solving:

$$(\Phi^T \Phi)w = \Phi^T y$$

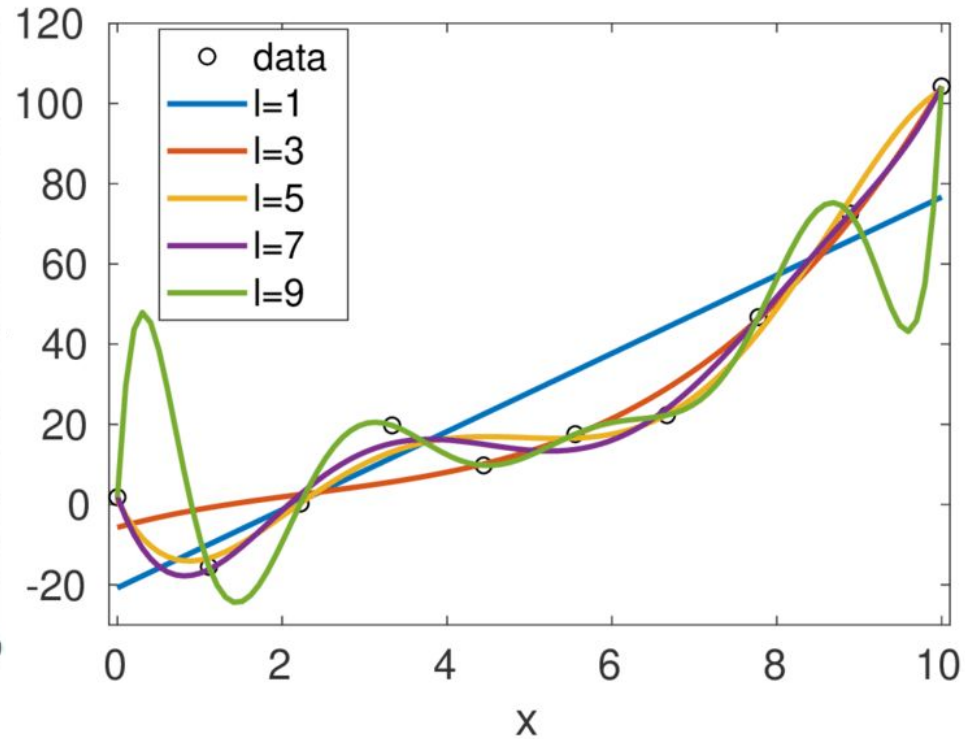
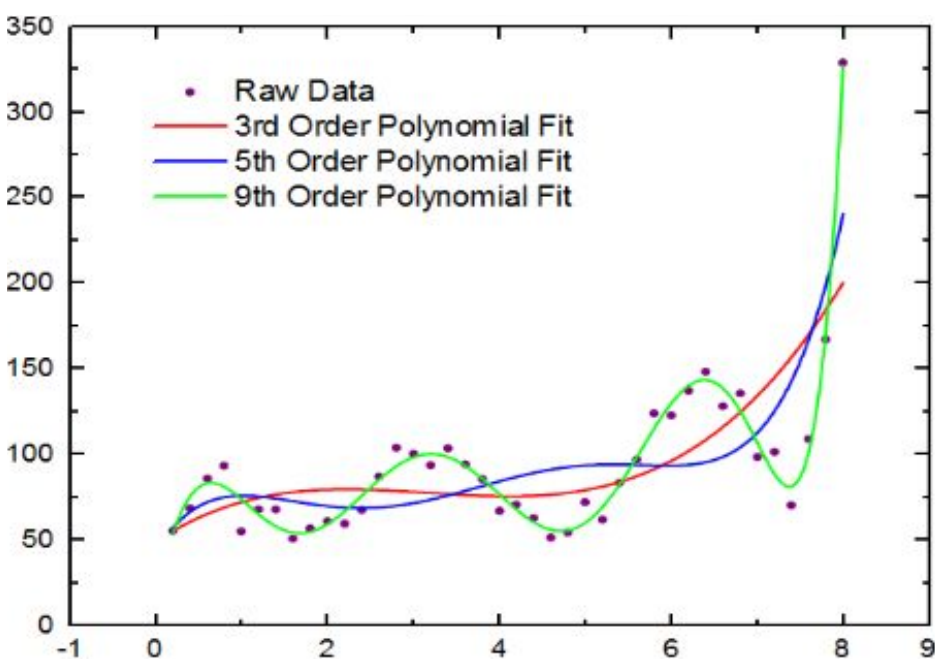
If  $\kappa(\Phi^T \Phi)$  is large:

- Small noise in  $y$
- Small rounding error
- Tiny perturbation in data  $\Rightarrow$  Huge change in  $w$

Model becomes unstable.

Ill-conditioning  $\neq$  overfitting. It is a numerical stability issue, but high-degree polynomials often cause both.

# Illustration



# Polynomial Regression Model

## Simple case of polynomial regression:

Expanding on the formula given on slide 12, choosing  $h_m(X) = X^m$  we get the model of:

$$f(X) = \sum_{m=0}^{M-1} \beta_m h_m(X) = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_{M-1} x^{M-1} + \epsilon$$

Let's look over at the **Hardwood data**. We have 19 observations concerning the strength of kraft paper (y) and the percentage of hardwood (x) in the batch of pulp from which the paper was produced.

Three polynomial models along with the linear model were fitted to the data. The summary statistics of the four models are reported below: Which model is the best?

| Order of poly. model | $R^2$  | $R^2_{\text{adj}}$ |
|----------------------|--------|--------------------|
| 1 (linear model)     | 0.3054 | 0.2645             |
| 2 (quadratic model)  | 0.9085 | 0.8971             |
| 3 (cubic model)      | 0.9707 | 0.9648             |

# Types of Polynomials

Linear —————  $ax + b = 0$

Quadratic —————  $ax^2 + bx + c = 0$

Cubic —————  $ax^3 + bx^2 + cx + d = 0$

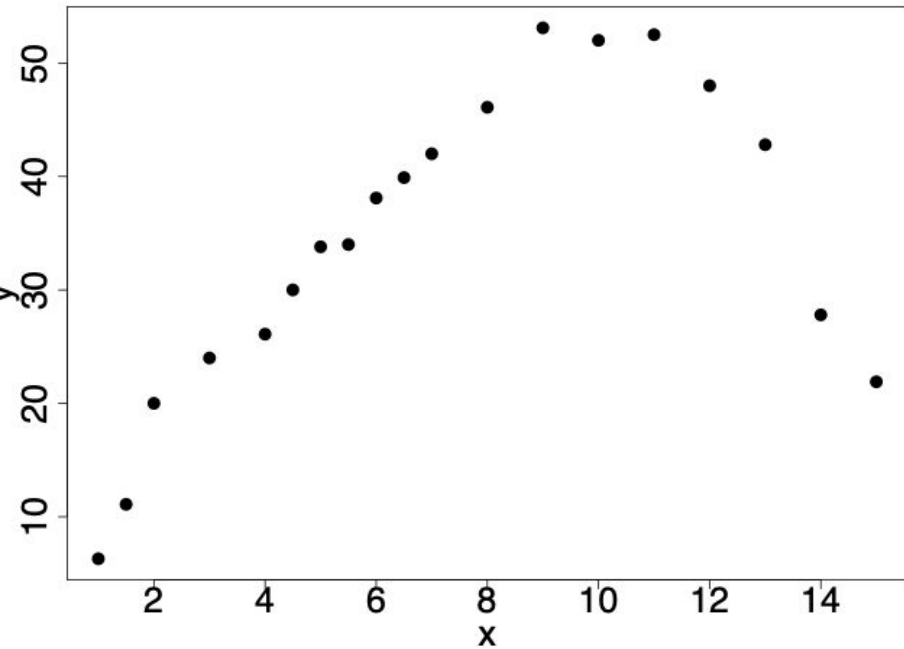
$$y = b_0 + b_1x_1$$

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

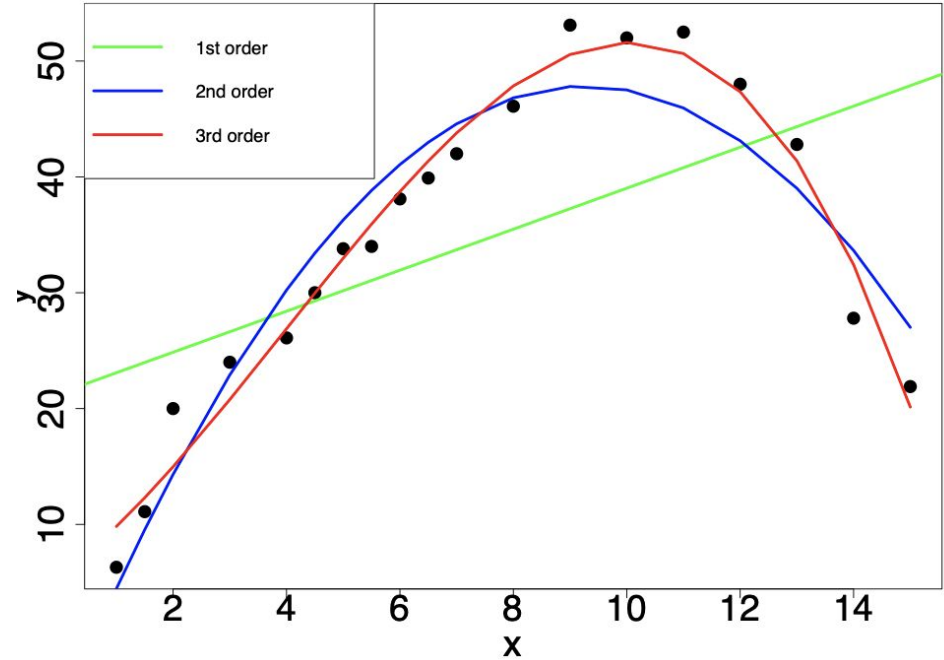
$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{bmatrix},$$

# Illustration



**scatterplot of hardwood data**



# Key points

## Key points in building the model:

### → Order of the model:

We can always get a perfect model using a polynomial of the  $(n-1)th$  degree, however such model is extremely overfit and doesn't hold much predictive power. We can use two strategies to fit:

- ◆ **Forward selection:** start with linear, then add a quadratic components... Repeat until the loss doesn't improve significantly.
- ◆ **Backward elimination:** Start with the  $(n-1)th$  degree polynomial and reduce until the term has significant t statistic-the weight of the term has significant impact(is not close to 0).

### → ILL-conditioning:

A basic assumption in linear regression analysis is that rank of X-matrix is full column rank. In the setting of polynomial regression, the design matrix may have lots of columns corresponding to just one predictor or two. Those columns will have a significant multicollinearity, especially when the values of x are limited to a narrow range.

As a result, the  $(X'X)^{-1}$  may not be accurate, and parameters will be estimated with a considerable error. Centering the data may remove some nonessential ill-conditioning.

# Key points

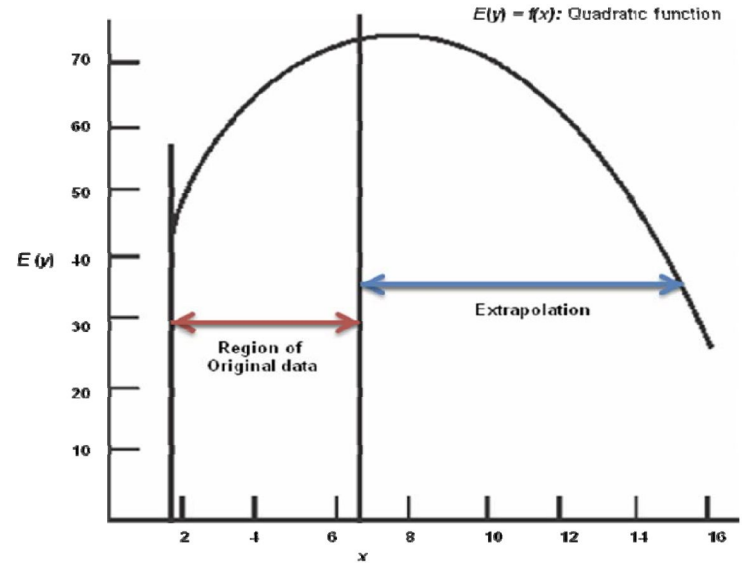
## Key points in building the model:

### → Extrapolation:

Because polynomial models may turn in unanticipated and inappropriate directions, extrapolation with them can be extremely hazardous.

### → Hierarchy

If the  $n$ th degree polynomial contains all the terms up to the  $n$ th degree, it's called hierarchical. It is expected that all polynomial models should have this property because only hierarchical models are invariant under linear transformation.



# Runge Phenomenon

High-degree polynomial interpolation may oscillate wildly near boundaries.

Even if training error is zero.

## Why?

Polynomial basis functions are global:  $x^p$ . Changing one coefficient affects the entire domain. There is no locality.

## Geometric View

As degree increases,  $\text{Col}(\Phi)$  enlarges and projection fits data exactly.

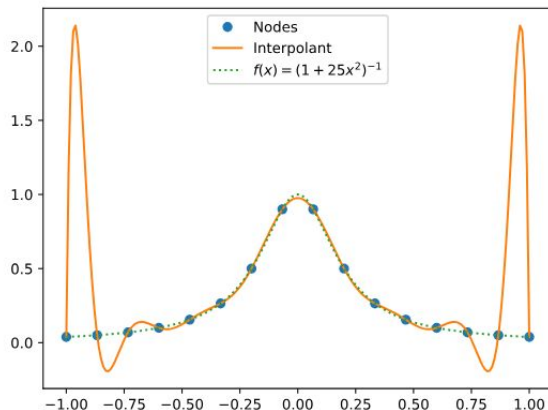
But between data points, function may fluctuate dramatically.



# Runge Phenomenon

When fitting high-degree polynomial:

- System tries to pass through all points.
- Constraints force large coefficients.
- Coefficients alternate in sign.
- Boundary regions amplify oscillation.



# Hierarchical Models

A polynomial model is hierarchical if:

If degree  $p$  term is included, all lower-degree terms are also included.

Example:

$$f(x) = w_0 + w_1x + w_2x^2 + w_3x^3$$

Valid (hierarchical).

Not hierarchical:

$$f(x) = w_0 + w_3x^3$$

Missing lower-order terms.

# Hierarchical Models

## Why Hierarchy Matters

Suppose we shift input:

$$z = x + c$$

Then:

$$x^3 = (z - c)^3$$

Expands into:

$$z^3 - 3cz^2 + 3c^2z - c^3$$

If model lacks lower-degree terms:

It cannot represent the same function after shift.

# Colab link

[Google Colab Practical demo.](#)

# Regularization

## Main Principle:

To control over-fitting, a regularization term to the error function can be introduced in the form of:

$$E_D(\mathbf{w}) + \lambda E_W(\mathbf{w})$$

where  $\lambda$  is the regularization coefficient that controls the relative importance of the data-dependent error  $E_D(w)$  and the regularization term  $E_W(w)$ . One of the simplest forms of regularizer is given as:

$$E_W(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

If we also consider the sum-of-squares error function given by:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2$$

then the total error function becomes:

$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}.$$

# Regularization

## Main Principle:

This choice of regularizer is known as **weight decay**, as in sequential algorithms it encourages weight values to decay towards zero, unless supported by data. In statistics, it shrinks parameter values towards zero. It has the advantage that the error function remains a quadratic function of  $\mathbf{w}$ , and so its exact minimizer can be found in closed form. Specifically, setting the gradient of with respect to  $\mathbf{w}$  to zero, and solving for  $\mathbf{w}$  as before, we obtain:

$$\mathbf{w} = (\lambda \mathbf{I} + \Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

This represents a simple extension of the least-squares solution.

A more general regularizer is sometimes used, for which the regularized error takes the form of:

$$\frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^T \phi(\mathbf{x}_n)\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q$$

Where  $q = 2$  corresponds to the quadratic regularizer. The case of  $q = 1$  is known as the **lasso**, if  $\lambda$  is sufficiently large, some of the coefficients  $w_j$  are driven to zero, leading to a sparse model in which the corresponding basis functions play no role.